

Trabalho Prático Final – Entrega Parcial

User Stories e Cartões CRC

1. Integrantes do grupo 7

#	Nome completo	Matrícula
1	Denyse Freitas Santos	124693
2	Gustavo Lopes Pimenta	124684
3	Luisa Sousa Sarti	124666
4	Pedro Henrique Marques Lucas	124682

2. Tema

Jogo no estilo tower defense, semelhante a *Plants vs. Zombies*, com temática da UFV. O jogador posiciona personagens (as “torres”) em um tabuleiro para defender o núcleo da base contra ondas de inimigos que se tornam gradualmente mais fortes e ameaçam destruí-lo.

Ambientação (proposta inicial, sujeita a refinamento): os defensores representam figuras da vida universitária (estudante de Informática, professor de Cálculo, monitor, cozinheiro do RU) e os inimigos representam as “ameaças” do semestre (prova surpresa, prazo de entrega, reprovação). O recurso do jogo são as fichas do RU.

Modos de operação

- **Modo interativo (GUI):** partida em tempo real, com o jogador escolhendo e posicionando defensores com o mouse.
- **Modo headless (simulação em lote):** lê de um arquivo de entrada a configuração da fase e os comandos (posicionamentos e avanço de tempo), executa a partida sem interface e grava um arquivo de saída com o log e as estatísticas finais.

3. User Stories

Formato: Como [quem] eu quero [o quê] para [por quê], seguido dos critérios de aceitação.

US01 – Posicionar defensor

Como jogador, **eu quero** posicionar um defensor em uma célula livre do tabuleiro **para** montar minha estratégia de defesa.

Critérios de aceitação

- Só é possível posicionar se houver fichas suficientes; o custo é debitado ao posicionar
- Célula ocupada não aceita novo defensor e o jogo informa o motivo
- O defensor aparece imediatamente na célula escolhida
- É possível cancelar a seleção antes de confirmar

US02 – Escolher entre tipos de defensor

Como jogador, **eu quero** escolher entre diferentes tipos de defensor, cada um com função própria **para** adaptar a estratégia aos inimigos que enfrento.

Critérios de aceitação

- Existem ao menos quatro tipos (ataque, dano forte, suporte e geração de recursos)
- Custo e descrição de cada tipo são exibidos antes da escolha
- Um tipo em tempo de recarga aparece como indisponível
- O comportamento de cada tipo é distinguível durante a partida

US03 – Gerenciar fichas do RU

Como jogador, **eu quero** acumular fichas do RU ao longo da partida **para** comprar novos defensores.

Critérios de aceitação

- O saldo de fichas está sempre visível
- As fichas aumentam por defensores de geração de recursos e por inimigos derrotados
- O saldo nunca fica negativo
- Ganhos e gastos são refletidos imediatamente no saldo

US04 – Enfrentar ondas progressivas

Como jogador, **eu quero** enfrentar ondas de inimigos cada vez mais fortes **para** sentir a progressão de dificuldade ao longo da partida.

Critérios de aceitação

- A onda atual e o total de ondas são indicados
- A cada onda aumentam a quantidade, a resistência ou a velocidade dos inimigos
- Há inimigos de tipos diferentes, com características próprias
- O jogador é avisado antes de uma onda especial (chefe)

US05 – Acompanhar a vida do núcleo

Como jogador, **eu quero** acompanhar a vida do núcleo **para** saber o quão perto estou de perder.

Critérios de aceitação

- A vida do núcleo é exibida de forma contínua
- A vida diminui quando um inimigo o alcança
- Com vida zero, a partida termina em derrota com mensagem clara
- Superar a última onda com o núcleo vivo termina a partida em vitória

US06 – Pausar, retomar e reiniciar

Como jogador, **eu quero** pausar, retomar e reiniciar a partida **para** jogar no meu próprio ritmo.

Critérios de aceitação

- A pausa congela inimigos, projéteis, recargas e tempo
- Ao retomar, a partida continua exatamente do mesmo estado
- Reiniciar volta a fase ao estado inicial, após confirmação.

US07 – Escolher fase e dificuldade

Como jogador, **eu quero** escolher uma fase com cenário do campus e nível de dificuldade **para** variar o desafio a cada partida.

Critérios de aceitação

- As fases são listadas com nome e dificuldade
- Cada fase tem parâmetros próprios (ondas, saldo inicial, vida do núcleo, defensores disponíveis)
- A fase escolhida é carregada com os parâmetros corretos

US08 – Executar simulação em lote (headless)

Como avaliador do sistema, **eu quero** executar uma partida em lote a partir de um arquivo de entrada, sem interface **para** reproduzir e testar cenários de forma automática.

Critérios de aceitação

- O caminho do arquivo é informado ao iniciar o programa
- Todos os comandos do arquivo (posicionamentos e avanço de tempo) são processados sem interação
- O mesmo arquivo de entrada sempre produz a mesma saída
- O programa termina sozinho ao fim da partida, sem travar

US09 – Gerar arquivo de saída com log e estatísticas

Como avaliador do sistema, **eu quero** receber um arquivo de saída com o log e as estatísticas da partida **para** analisar o desempenho da defesa.

Critérios de aceitação

- O log registra os eventos em ordem cronológica
- O resumo final traz resultado, ondas superadas, inimigos derrotados, fichas gastas e vida restante do núcleo
- O arquivo é gerado mesmo quando a partida termina em derrota.

US10 – Receber mensagens claras em caso de erro

Como usuário do sistema, **eu quero** receber mensagens claras quando algo inválido acontece **para** corrigir o problema sem que o programa pare de funcionar.

Critérios de aceitação

- Arquivo inexistente ou mal formatado informa o motivo e a linha do problema
- Ações inválidas no modo interativo (ex.: posição inválida) não encerram o jogo
- No modo headless, um comando inválido é registrado no log

Possíveis extras: efeitos sonoros, placar com melhores pontuações, salvamento de progresso.

4. Cartões CRC (Classe – Responsabilidade – Colaboração)

As classes foram identificadas a partir dos substantivos das User Stories e as responsabilidades dos verbos. Responsabilidades de conhecimento (dados que a classe mantém) e de realização (o que ela faz) aparecem separadas por tipo. Classes marcadas como abstratas definem o contrato que as subclasses implementam (polimorfismo).

4.1 Controle da partida

Classe: Partida	
Responsabilidades	Colaborações
Conhece: - Estado atual (em andamento, pausada, vitória, derrota) e tempo decorrido Faz: - Executar o ciclo de atualização do jogo - Pausar, retomar e reiniciar - Decidir vitória e derrota - Orquestrar as demais classes	- Tabuleiro - GerenciadorDeOndas - Economia - Nucleo - Fase - Relatorio

Classe: Fase	
Responsabilidades	Colaborações
Conhece: - Dimensões do tabuleiro, saldo inicial, vida do núcleo, defensores disponíveis e cronograma de ondas Faz: - Fornecer seus parâmetros para montar a partida	- LeitorDeCenario - GerenciadorDeOndas

Classe: GerenciadorDeOndas	
Responsabilidades	Colaborações
Conhece: - Cronograma das ondas (tipos, quantidades e intervalos) - Onda atual Faz: - Gerar inimigos no momento certo - Aumentar a dificuldade a cada onda - Informar o fim de cada onda e da última onda	- FabricaDeEntidades - Inimigo - Tabuleiro - Fase

Classe: Economia	
Responsabilidades	Colaborações
Conhece: - Saldo atual de fichas do RU Faz: - Creditar fichas - Debitar fichas - Verificar se há saldo suficiente (lança SaldoInsuficienteException)	- Defensor - Inimigo - Partida

Classe: Nucleo	
Responsabilidades	Colaborações
Conhece: - Vida atual e vida máxima Faz: - Receber dano - Informar se foi destruído	- Inimigo - Partida

4.2 Tabuleiro e entidades

Classe: Tabuleiro	
Responsabilidades	Colaborações
Conhece: - Grade de linhas x colunas - Ocupante de cada célula - Inimigos presentes em cada linha Faz: - Validar e registrar posicionamento (lança PosicaoInvalidaException) - Remover entidades destruídas - Informar inimigos e defensores de uma linha	- Defensor - Inimigo - Projetoil

Classe: Entidade (abstrata)	
Responsabilidades	Colaborações
Conhece: - Posição - Vida atual Faz: - Atualizar-se a cada passo de tempo (método virtual puro) - Receber dano - Informar se está viva	Tabuleiro

Classe: Defensor (abstrata, herda de Entidade)	
Responsabilidades	Colaborações
Conhece: - Custo - Tempo de recarga - Alcance - Descrição Faz: - Executar sua ação própria a cada passo de tempo (polimorfismo) - Informar se está disponível para compra	- Projetoil - Inimigo - Economia - Tabuleiro

Classe: Subclasses de Defensor (herdam de Defensor)	
Responsabilidades	Colaborações
Conhece: - EstudanteDeInformatica: dispara projéteis na própria linha - ProfessorDeCalculo: causa dano elevado, com recarga longa - MonitorDaTurma: fortalece ou cura defensores vizinhos - CozinheiroDoRU: gera fichas periodicamente Faz: - Cada subclasse implementa a sua versão da ação do defensor	- Projetoil - Economia - Tabuleiro

Classe: Inimigo (abstrata, herda de Entidade)	
Responsabilidades	Colaborações
Conhece: - Velocidade - Dano por ataque - Recompensa em fichas Faz: - Mover-se em direção ao núcleo - Atacar o defensor que o bloqueia ou o núcleo - Ao ser derrotado, creditar a recompensa	- Tabuleiro - Defensor - Núcleo - Economia

Classe: Subclasses de Inimigo (herdam de Inimigo)	
Responsabilidades	Colaborações
Conhece: - ProvaSurpresa: rápida e frágil - PrazoDeEntrega: equilíbrio entre velocidade e resistência - Reprovacao: lenta, muito resistente (chefe) Faz: - Cada subclasse define seus atributos e seu comportamento de movimento/ataque	- Tabuleiro - Defensor - Nucleo

Classe: Projétil	
Responsabilidades	Colaborações
Conhece: - Posição - Velocidade - Dano Faz: - Mover-se pela linha - Detectar colisão e causar dano ao inimigo atingido - Ser removido após o impacto ou ao sair do tabuleiro	- Inimigo - Tabuleiro

Classe: FabricaDeEntidades	
Responsabilidades	Colaborações
Conhece: - Associação entre identificadores textuais e tipos de Defensor/Inimigo Faz: - Criar o defensor ou inimigo correto a partir de um identificador (usado pela interface e pelos arquivos de entrada)	- Defensor - Inimigo - GerenciadorDeOndas - LeitorDeCenario

4.3 Modos de operação, arquivos e erros

Classe: ModoInterativo (interface gráfica)	
Responsabilidades	Colaborações
Conhece: - Janela, elementos visuais e entradas do usuário Faz: - Desenhar tabuleiro, entidades e informações do jogo (fichas, onda, vida do núcleo) - Capturar cliques e converter em comandos do jogador - Exibir mensagens de erro ao usuário	- Partida - Tabuleiro - FabricaDeEntidades - Economia

Classe: ModoHeadless (simulação em lote)	
Responsabilidades	Colaborações
Conhece: - Caminhos dos arquivos de entrada e de saída Faz: - Carregar o cenário e a lista de comandos do arquivo - Executar os comandos e avançar o tempo sem interface - Gravar o relatório ao final	- LeitorDeCenario - Partida - Relatorio

Classe: LeitorDeCenario	
Responsabilidades	Colaborações
Conhece: • Formato do arquivo de entrada Faz: • Ler e validar o arquivo • Construir a Fase e a lista de comandos (lança ArquivoInvalidoException)	• Fase • FabricaDeEntidades • ModoHeadless

Classe: Relatorio	
Responsabilidades	Colaborações
Conhece: • Eventos ocorridos e estatísticas (inimigos derrotados, ondas superadas, fichas gastas, vida restante, resultado) Faz: • Registrar eventos em ordem cronológica • Gerar o resumo final • Gravar o arquivo de saída	• Partida • ModoHeadless

Classe: ExcecaoDoJogo (hierarquia de exceções)	
Responsabilidades	Colaborações
Conhece: • Mensagem descritiva do erro • Subclasses: PosicaoInvalidaException, SaldoInsuficienteException, ArquivoInvalidoException Faz: • Sinalizar situações de erro de forma específica para cada caso, permitindo tratamento adequado	• Tabuleiro • Economia • LeitorDeCenario • ModoInterativo • ModoHeadless